

实验 2 循环神经网络 RNN 实验

一、实验目的

学习掌握循环神经网络（RNN）的基本原理及 LSTM 的基本结构；
掌握利用 LSTM 神经元构造循环神经网络进行训练和预测时间序列。

二、实验内容

利用 python 语言搭建基于 LSTM 的 RNN 模型。

利用 RNN 模型对正弦曲线或余弦曲线的数值变化进行预测，所用到的数据集由学生自己利用 python 编写代码采样生成，目标是利用正弦曲线或者余弦曲线的前 3 个点的数值预测下一个点的数值。扩展实验是使用 GRU 网络与 LSTM 网络进行比较，使用多层 RNN 来进行预测，对预测结果进行对比分析。实验要求大家掌握 RNN 模型，并进一步理解 LSTM 和 GRU 模型。

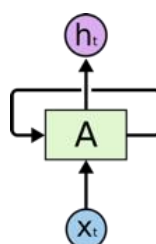
三、实验环境

硬件：x86_64 Centos 3.10.0 服务器/GPU 服务器、GPU、PC 上位机

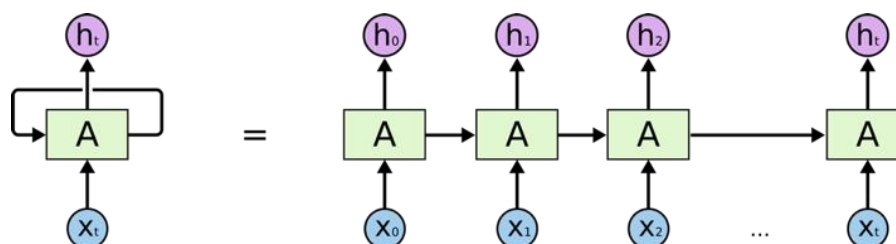
软件：python3.5、torch1.3, numpy1.12.1

四、实验原理

RNN 是一种用于处理时序数据的神经网络模型。在传统神经网络中，模型不会关注上一时刻的处理会有什么信息可以用于下一时刻，每一次都只会关注当前时刻的处理。举个例子来说，我们想对一部影片中每一刻出现的事件进行分类，如果我们知道电影前面的事件信息，那么对当前时刻事件的分类就会非常容易。实际上，传统神经网络没有记忆功能，所以它对每一刻出现的事件进行分类时不会用到影片已经出现的信息。基于对这种时间序列关注的需求，循环神经网络应运而生。递归神经网络的结果与传统神经网络有一些不同，它带有一个指向自身的环，用来表示它可以传递当前时刻处理的信息给下一时刻使用，结构如下：

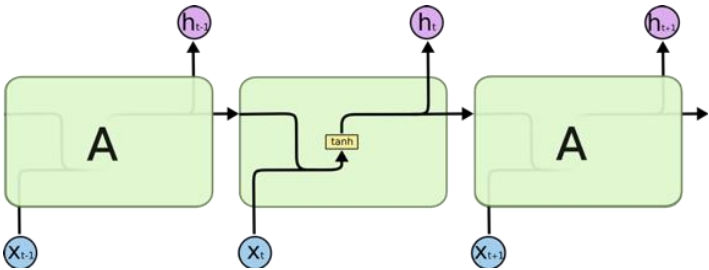


其中， x_t 为时间节点的输入，A 为模型处理部分， h_t 为 t 时间节点的输出。为了更好地说明 RNN 的工作原理，可以将上图的循环网络结构展开，得到：

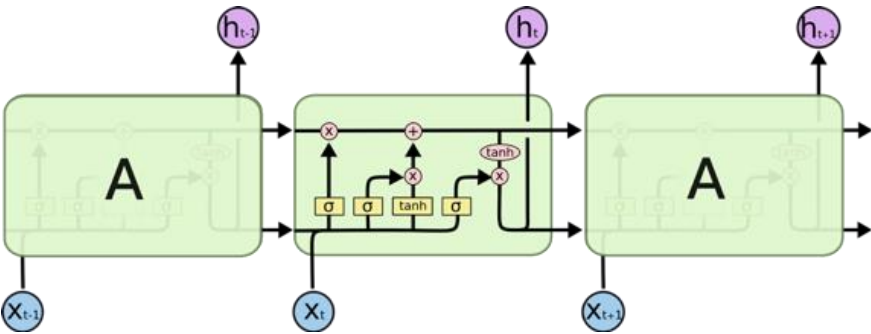


这样的一条链状神经网络代表了一个递归神经网络，可以认为它是对相同神经网络的多重复制，每一时刻的神经网络会传递信息给下一时刻。在本次试验中可以认为 X_i 为当前 i 时的正弦曲线值， h_i 为预测的 $i+1$ 时刻的正弦曲线值。

不过 RNN 也有它自己的问题，即在当预测点与依赖的相关信息距离比较远的时候，就难以学到该相关信息。为了解决这类问题，人们发明了 Long Short Term Memory (LSTM)。其结构与普通的循环神经网络的神经元结构对比如下：

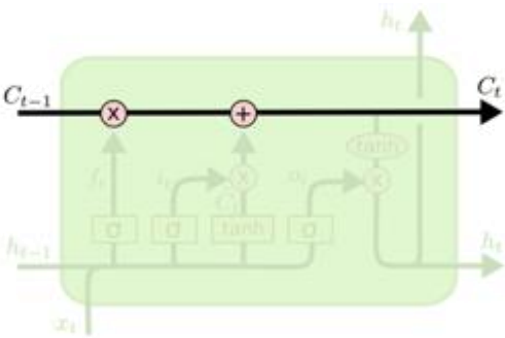


普通RNN

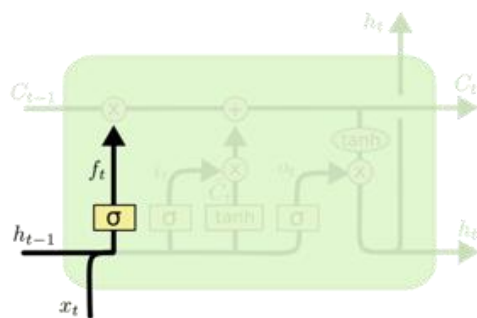


基于LSTM的RNN

理解 LSTMs 的关键就是下面的矩形方框，被称为 memory block (记忆块)，主要包含了三个门 (forget gate、input gate、output gate) 与一个记忆单元 (cell)。方框内上方的那条水平线，被称为 cell state (单元状态)，它就像一个传送带，可以控制信息传递给下一时刻。



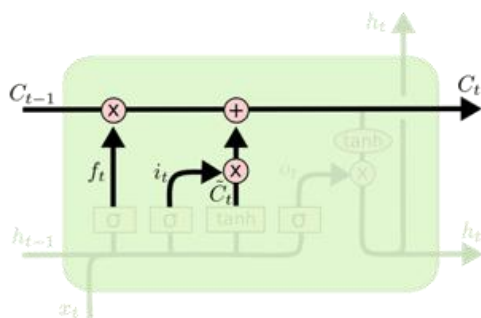
下面来逐步了解 LSTM 的工作原理。LSTM 第一步是用来决定什么信息可以通过 cell state。这个决定由“forget gate”层通过 sigmoid 来控制，它会根据上一时刻的输出 $h_{(t-1)}$ 和当前输入 x_t 来产生一个 0 到 1 的 f_t 值，来决定是否让上一时刻学到的信息 $C_{(t-1)}$ 通过或部分通过。如下：



$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

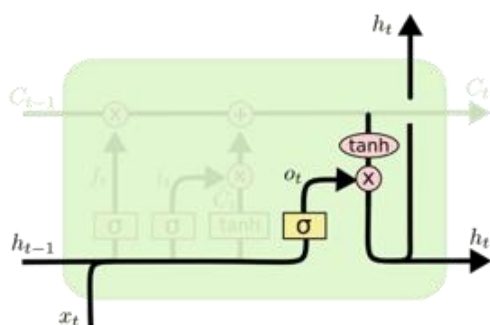
举个例子来说就是，我们在之前的句子中学到了很多东西，一些东西对当前来讲是没用的，可以对它进行选择性地过滤。

第二步是产生我们需要更新的新信息。这一步包含两部分，第一个是一个“input gate”层通过 sigmoid 来决定哪些值用来更新，第二个是一个 tanh 层用来生成新的候选值 $\tilde{C}_t$ ，它作为当前层产生的候选值可能会添加到 cell state 中。我们会把这两部分产生的值结合来进行更新。



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

最后一步是决定模型的输出，首先是通过 sigmoid 层来得到一个初始输出，然后使用 tanh 将 C_t 值缩放到 -1 到 1 间，再与 sigmoid 得到的输出逐对相乘，从而得到模型的输出。



$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

五、实验步骤

提示：实验步骤中给出的代码不是按照代码顺序给出，请结合代码截图中的行号进行实验。

1、下面开始实验，给定 sin 函数的前三个值，预测后一个值。启动 jupyter，引入需要的包。

```
1. import numpy as np
2. import pandas as pd
3. import matplotlib.pyplot as plt
4. import math
```

```

5. import pdb
6. import torch
7. from torch import nn
8. from torch.autograd import Variable

```

- 2、首先进行数据集的创建，首先利用 numpy 或 math 工具包进行正弦值或者余弦值序列的生成，并划分训练集和测试集的范围。

```

1. dataset=[]
2. for data in np.arange(0, 3, .01):
3.     data = math.sin(data*math.pi)
4.     dataset.append(data)
5. dataset=np.array(dataset)
6. dataset = dataset.astype('float32')
7. max_value = np.max(dataset)
8. min_value = np.min(dataset)
9. scalar = max_value - min_value
10. dataset = list(map(lambda x: x / scalar, dataset))
11.
12. def create_dataset(dataset, look_back=3):
13.     dataX, dataY = [], []
14.     for i in range(len(dataset) - look_back):
15.         a = dataset[i:(i + look_back)]
16.         dataX.append(a)
17.         dataY.append(dataset[i + look_back])
18.     return np.array(dataX), np.array(dataY)
19.
20. data_X, data_Y = create_dataset(dataset)

```

- 3、对数据进行 reshape，同时转化成 tensor。

```

1. train_X = train_X.reshape(-1, 1, 3)
2. train_Y = train_Y.reshape(-1, 1, 1)
3. test_X = test_X.reshape(-1, 1, 3)
4.
5. train_x =    ##补充代码，将 numpy 转化为 tensor
6. train_y =    ##补充代码，将 numpy 转化为 tensor
7. test_x =     ##补充代码，将 numpy 转化为 tensor

```

- 4、编写 RNN 模型，主要包括 RNN 模型和一层全连接模型，全连接模型用于预测模型最终输出值。由于 RNN 和全连接模型的输入输出 shape 不一样，中间需要用到一些 reshape 操作。RNN 模型超参数含义参考官方文档：

<https://pytorch.org/docs/stable/nn.html?highlight=rnn#torch.nn.RNN>

```

1. class RNN(nn.Module):

```

```

2.     def __init__(self, input_size, hidden_size, output_size=1, num_layers=2):
3.         super(RNN, self).__init__()
4.
5.         self.rnn =      # 补充代码 rnn
6.         self.linear =   # 补充代码全连接
7.
8.     def forward(self, x):
9.         # 补充代码 forward 函数
10.        # 补充代码 forward 函数
11.        return x

```

5、定义优化器和损失函数。损失函数使用最小均方误差函数，这个函数经常用于回归模型，优化器采用 adam 优化器。

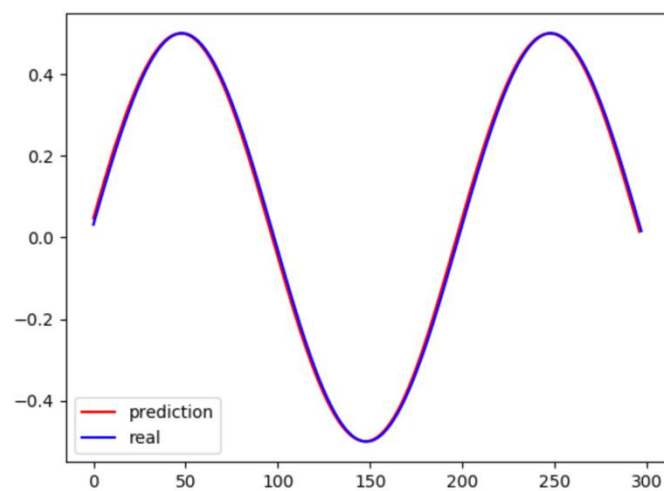
```

1. net = RNN(3, 20)
2.
3. criterion = nn.MSELoss()
4. optimizer = torch.optim.Adam(net.parameters(), lr=1e-2)
5. # 开始训练
6. for e in range(1000):
7.     var_x = Variable(train_x)
8.     var_y = Variable(train_y)
9.     ##### 补充代码，前向传播
10.    ##### 补充代码，计算 loss
11.    optimizer.zero_grad()
12.    ##### 补充代码，反向传播
13.    if (e + 1) % 100 == 0: # 每 100 次输出结果
14.        print('Epoch: {}, Loss: {:.10f}'.format(e + 1, loss.data.item()))

```

6、进行预测，并对结果进行可视化。示范预测结果如下图所示，蓝色线为真实值，红色线为预测值。

7、



六、后续实验

以上是利用单层 RNN 使用 3 个连续的正弦函数值或者余弦函数值来预测第 4 个值的实验。在后续实验中，（1）使用 GRU 网络与 LSTM 网络进行比较。（2）使用多层 RNN 来进行预测，对预测结果进行对比分析。